

Verteilte Systeme, Containerisierung

Vorwort:

Grundlage für diese Aufgabe ist die Präsentation «Docker Refresher»

Dauer: 100 Minuten

Besprechung: 20 Minuten

Total: 120 Minuten

In diesem Refresher wiederholen Sie Docker, Docker Compose und Docker Images praktisch. Ziel ist, dass die wichtigsten Befehle, Begriffe und Fehlerbilder wieder sitzen, bevor der Kurs zu Kubernetes wechselt.

Offizielle Dokumentation

<https://docs.docker.com/>

<https://docs.docker.com/compose/>

<https://docs.docker.com/build/building/best-practices/>

Aufgabe 1)

Docker Mental Model

Ordnen Sie zuerst die wichtigsten Begriffe. Arbeiten Sie bewusst kurz und präzise.

a) Erklären Sie die Begriffe Image, Container, Dockerfile, Registry, Compose und Kubernetes in je einem Satz.

- ❓ **Image:** Vorlage eines Containers mit Applikation, Libraries und Abhängigkeiten.
- ❓ **Container:** Laufende Instanz eines Images.
- ❓ **Dockerfile:** Datei mit Anweisungen zum Erstellen eines Images.
- ❓ **Registry:** Speicherort für Images (z.B. Docker Hub).
- ❓ **Compose:** Tool zum Definieren und Starten mehrerer Container.
- ❓ **Kubernetes:** Orchestrierungssystem zur Verwaltung von Containern im Cluster.

b) Beschreiben Sie den Weg von einem Image zu einem laufenden Container bis zur Fehlersuche mit Logs.

Dockerfile → docker build → Image
Image wird in Registry gespeichert oder lokal genutzt
docker run → Container starten
Container läuft → docker ps prüfen
Fehleranalyse → docker logs <container>

c) Nennen Sie zwei Dinge, die Container erleichtern, und zwei Dinge, die Container nicht automatisch lösen.

- ✅ Erleichtern:
 - Portabilität (läuft überall gleich)

- Isolation von Anwendungen

✗ Lösen nicht:

- Monitoring / Logging automatisch
- Skalierung / Orchestrierung (brauchst Kubernetes)

Aufgabe 2)

Container-Check mit NGINX

Starten, untersuchen und entfernen Sie einen einzelnen Container sauber.

a) Starten Sie einen NGINX-Container im Hintergrund mit dem Namen «refresh-web» und Port-Mapping 8080:80. Notieren Sie den Befehl.

Tipp: docker run, -d, --name, -p

```
docker run -d --name refresh-web -p 8080:80 nginx
```

b) Prüfen Sie Status, Container-ID und Port-Mapping. Welche Ausgabe ist für Sie relevant?

Tipp: docker ps / docker inspect

```
docker inspect refresh-web
[
  {
    "Id": "206eb4774c16b19f392af5aac180c0f924ff5d66bdbbc70cd0cefc2e4df8d0d30",
    "Created": "2026-06-22T17:44:41.671106838Z",
    "Path": "/docker-entrypoint.sh",
    "Args": [
      "nginx",
      "-g",
      "daemon off;"
    ],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 604,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2026-06-22T17:44:44.452246735Z",
      "FinishedAt": "0001-01-01T00:00:00Z"
    },
    "Image": "sha256:42f2d24ae18df9b5251d1cc45548085656d2335e9338fd150a24e415462d151f",
    "ResolvConfPath":
"/var/lib/docker/containers/206eb4774c16b19f392af5aac180c0f924ff5d66bdbbc70cd0cefc2e4df8d0d30/resolv.conf",
    "HostnamePath":
"/var/lib/docker/containers/206eb4774c16b19f392af5aac180c0f924ff5d66bdbbc70cd0cefc2e4df8d0d30/hostname",
    "HostsPath":
"/var/lib/docker/containers/206eb4774c16b19f392af5aac180c0f924ff5d66bdbbc70cd0cefc2e4df8d0d30/hosts",

```

```
"LogPath":
"/var/lib/docker/containers/206eb4774c16b19f392af5aac180c0f924ff5d66bdbbc70cd0cefc2e4df8d0d30/206e
b4774c16b19f392af5aac180c0f924ff5d66bdbbc70cd0cefc2e4df8d0d30-json.log",
"Name": "/refresh-web",
"RestartCount": 0,
"Driver": "overlayfs",
"Platform": "linux",
"MountLabel": "",
"ProcessLabel": "",
"AppArmorProfile": "",
"ExecIDs": null,
"HostConfig": {
  "Binds": null,
  "ContainerIDFile": "",
  "LogConfig": {
    "Type": "json-file",
    "Config": {}
  },
  "NetworkMode": "bridge",
  "PortBindings": {
    "80/tcp": [
      {
        "HostIp": "",
        "HostPort": "8080"
      }
    ]
  },
  "RestartPolicy": {
    "Name": "no",
    "MaximumRetryCount": 0
  },
  "AutoRemove": false,
  "VolumeDriver": "",
  "VolumesFrom": null,
  "ConsoleSize": [
    14,
    203
  ],
  "CapAdd": null,
  "CapDrop": null,
  "CgroupnsMode": "host",
  "Dns": null,
  "DnsOptions": [],
  "DnsSearch": [],
  "ExtraHosts": null,
  "GroupAdd": null,
  "IpcMode": "private",
  "Cgroup": "",
```

```
"Links": null,
"OomScoreAdj": 0,
"PidMode": "",
"Privileged": false,
"PublishAllPorts": false,
"ReadonlyRootfs": false,
"SecurityOpt": null,
"UTSMode": "",
"UsersnsMode": "",
"ShmSize": 67108864,
"Runtime": "runc",
"Isolation": "",
"CpuShares": 0,
"Memory": 0,
"NanoCpus": 0,
"CgroupParent": "",
"BlkioWeight": 0,
"BlkioWeightDevice": [],
"BlkioDeviceReadBps": [],
"BlkioDeviceWriteBps": [],
"BlkioDeviceReadIOps": [],
"BlkioDeviceWriteIOps": [],
"CpuPeriod": 0,
"CpuQuota": 0,
"CpuRealtimePeriod": 0,
"CpuRealtimeRuntime": 0,
"CpusetCpus": "",
"CpusetMems": "",
"Devices": [],
"DeviceCgroupRules": null,
"DeviceRequests": null,
"MemoryReservation": 0,
"MemorySwap": 0,
"MemorySwappiness": null,
"OomKillDisable": false,
"PidsLimit": null,
"Ulimits": [],
"CpuCount": 0,
"CpuPercent": 0,
"IOMaximumIOps": 0,
"IOMaximumBandwidth": 0,
"MaskedPaths": [
  "/proc/acpi",
  "/proc/asound",
  "/proc/interrupts",
  "/proc/kcore",
  "/proc/keys",
  "/proc/latency_stats",
```

```
    "/proc/sched_debug",
    "/proc/scsi",
    "/proc/timer_list",
    "/proc/timer_stats",
    "/sys/devices/virtual/powercap",
    "/sys/firmware"
  ],
  "ReadonlyPaths": [
    "/proc/bus",
    "/proc/fs",
    "/proc/irq",
    "/proc/sys",
    "/proc/sysrq-trigger"
  ]
},
"GraphDriver": {
  "Data": null,
  "Name": ""
},
"Mounts": [],
"Config": {
  "Hostname": "206eb4774c16",
  "Domainname": "",
  "User": "",
  "AttachStdin": false,
  "AttachStdout": false,
  "AttachStderr": false,
  "ExposedPorts": {
    "80/tcp": {}
  },
  "Tty": false,
  "OpenStdin": false,
  "StdinOnce": false,
  "Env": [
    "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
    "NGINX_VERSION=1.31.2",
    "NJS_VERSION=0.9.9",
    "NJS_RELEASE=1~trixie",
    "ACME_VERSION=0.4.1",
    "PKG_RELEASE=1~trixie",
    "DYNPKG_RELEASE=1~trixie"
  ],
  "Cmd": [
    "nginx",
    "-g",
    "daemon off;"
  ],
  "Image": "nginx",
```

```
"Volumes": null,
"WorkingDir": "",
"Entrypoint": [
  "/docker-entrypoint.sh"
],
"OnBuild": null,
"Labels": {
  "maintainer": "NGINX Docker Maintainers \u003cdocker-maint@nginx.com\u003e"
},
"StopSignal": "SIGQUIT",
"StopTimeout": 1
},
"NetworkSettings": {
  "Bridge": "",
  "SandboxID": "605a6ec56bc9c8676aa882889139d644b939d1c6a0144118a83a23257f940998",
  "SandboxKey": "/var/run/docker/netns/605a6ec56bc9",
  "Ports": {
    "80/tcp": [
      {
        "HostIp": "0.0.0.0",
        "HostPort": "8080"
      },
      {
        "HostIp": "::",
        "HostPort": "8080"
      }
    ]
  },
  "HairpinMode": false,
  "LinkLocalIPv6Address": "",
  "LinkLocalIPv6PrefixLen": 0,
  "SecondaryIPAddresses": null,
  "SecondaryIPv6Addresses": null,
  "EndpointID": "",
  "Gateway": "",
  "GlobalIPv6Address": "",
  "GlobalIPv6PrefixLen": 0,
  "IPAddress": "",
  "IPPrefixLen": 0,
  "IPv6Gateway": "",
  "MacAddress": "",
  "Networks": {
    "bridge": {
      "IPAMConfig": null,
      "Links": null,
      "Aliases": null,
      "MacAddress": "e2:bc:27:9d:70:ed",
      "DriverOpts": null,

```

```
"GwPriority": 0,
"NetworkID": "4ae6ee2e9a582f3a676f127751f3494a553556ac13e00a76ff38b2dddfd75054",
"EndpointID": "cf7cd8fff213aabce86a03b0e0125adb5400e0672f74fffb08425590bb8c5b34",
"Gateway": "172.17.0.1",
"IPAddress": "172.17.0.2",
"IPPrefixLen": 16,
"IPv6Gateway": "",
"GlobalIPv6Address": "",
"GlobalIPv6PrefixLen": 0,
"DNSNames": null
}
},
"ImageManifestDescriptor": {
  "mediaType": "application/vnd.oci.image.manifest.v1+json",
  "digest": "sha256:c85f13d520611b398cf6ffb0f75c894173d238401b58b1fb9b989cd1149fbe58",
  "size": 2290,
  "annotations": {
    "com.docker.official-images.bashbrew.arch": "amd64",
    "org.opencontainers.image.base.digest":
"sha256:1275c5673a6135ff07b289ddafe4e2270dceb08eda14c0c69bb1b93ee25a9416",
    "org.opencontainers.image.base.name": "debian:trixie-slim",
    "org.opencontainers.image.created": "2026-06-17T22:44:17Z",
    "org.opencontainers.image.revision": "00348041cc12284266751457dcfc10b15645476a",
    "org.opencontainers.image.source": "https://github.com/nginx/docker-
nginx.git#00348041cc12284266751457dcfc10b15645476a:mainline/debian",
    "org.opencontainers.image.url": "https://hub.docker.com/_/nginx",
    "org.opencontainers.image.version": "1.31.2"
  },
  "platform": {
    "architecture": "amd64",
    "os": "linux"
  }
}
}
```

c) Öffnen Sie die Webseite im Browser. Welche URL verwenden Sie und woran erkennen Sie, dass der Container erreichbar ist?

geht

d) Öffnen Sie eine Shell im Container und schauen Sie sich das Verzeichnis der ausgelieferten Webseite an.

Tipp: docker exec -it refresh-web sh

```
docker exec -it refresh-web sh
cd /usr/share/nginx/html
ls

PS C:\Users\FlükigerKim> ^C
PS C:\Users\FlükigerKim> docker exec -it refresh-web sh
#
#
# cd /usr/share/nginx/html
# ls
50x.html index.html
```

e) Versuchen Sie einen zweiten Container auf demselben Host-Port zu starten. Was passiert und wie lösen Sie das Problem sauber?

```
port is already allocated

anderen Port wählen oder container stoppen
docker run -d -p 8081:80 nginx
```

f) Stoppen und löschen Sie alle Container aus dieser Aufgabe.

```
docker stop refresh-web
docker rm refresh-web
```

Aufgabe 3)

Compose Mini-Stack

Beschreiben Sie eine kleine Umgebung mit zwei Services als Compose-Datei. Aus diesem Content

```
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"
  redis:
    image: redis:alpine
```

a) Starten Sie die Umgebung. Welche Container, Netzwerke und Ports entstehen?

Tipp: docker compose up -d / docker compose ps / docker network ls

b) Zeigen Sie Logs für die gesamte Umgebung und danach nur für den Web-Service an. Was ist der Unterschied?

alles vs nur ein Service

c) Prüfen Sie, ob Redis antwortet. Was bedeutet die Antwort «PONG»?

Tipp: docker compose exec redis redis-cli PING

C:\docker-refresher\compose>docker compose exec redis redis-cli PING

PONG

➡ PONG = funktioniert

d) Ergänzen Sie ein benanntes Volume für Redis. Warum ist ein benanntes Volume etwas anderes als Daten im Container-Dateisystem?

Ein benanntes Volume speichert Daten ausserhalb des Containers dauerhaft.
Daten im Container-Dateisystem gehen normalerweise verloren, wenn der Container gelöscht wird.
Ein named volume bleibt bestehen und kann von neuen Containern wieder verwendet werden

```
redis:
  image: redis:alpine
  volumes:
    - redis-data:/data
```

```
volumes:
  redis-data:
```

e) Stoppen Sie die Umgebung einmal ohne und einmal mit Volume-Löschung. Welche Befehle verwenden Sie?

Tipp: docker compose down / docker compose down -v

```
docker compose down
docker compose down -v
```

Aufgabe 4)

Eigenes Image bauen

Erstellen Sie eine sehr kleine eigene Webseite und verpacken Sie diese als Image.

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
LABEL org.opencontainers.image.title="docker-refresh-web"
```

a) Erstellen Sie eine einfache «index.html». Welche sichtbare Information soll Ihre Seite enthalten?

Eine einfache HTML-Seite mit sichtbarem Text, z. B. Titel „Docker Refresher Kim“ und eine kurze Beschreibung.

b) Bauen Sie das Image mit dem Tag «docker-refresh-web:1.0.0». Notieren Sie den Befehl und die wichtigste Build-Ausgabe.

Das Image wird mit folgendem Command erstellt und getaggt:
docker build -t docker-refresh-web:1.0.0 .
Wichtige Build-Ausgabe: „Successfully built“ und „Successfully tagged“.

c) Starten Sie einen Container aus Ihrem eigenen Image und prüfen Sie die Webseite im Browser.

Ein Container wird aus dem Image gestartet und im Browser über <http://localhost:8080> geprüft; die eigene Webseite muss angezeigt werden.

d) Erstellen Sie eine «.dockerignore». Welche Dateien oder Ordner gehören nicht in den Build Context?

In die .dockerignore gehören Dateien und Ordner, die nicht ins Image sollen, z. B. .git, Logs, node_modules oder .env, um den Build-Kontext zu verkleinern.

e) Ändern Sie nur die «index.html» und bauen Sie erneut. Welche Layer werden wiederverwendet und warum?

Tipp: docker history / Build-Cache-Ausgabe

Beim erneuten Build werden alle unveränderten Layer wiederverwendet, insbesondere das Base-Image (FROM nginx:alpine).

Nur der Layer mit COPY index.html wird neu erstellt, da sich diese Datei geändert hat.

Warum:

Docker verwendet einen Build-Cache und vergleicht jede Anweisung im Dockerfile. Wenn sich eine Datei oder ein Befehl nicht geändert hat, wird der vorhandene Layer aus dem Cache übernommen, was den Build schneller macht.

```
docker history docker-refresh-web:1.0.0
```


Aufgabe 5)

Tags, Debugging und Registry-Readiness

Bereiten Sie Ihr Image so vor, dass es später eindeutig referenziert werden könnte. Ein Push in eine Registry ist hier nicht nötig.

a) Taggen Sie Ihr Image zusätzlich als «teko/docker-refresh-web:1.0.0» und «teko/docker-refresh-web:latest». Was ist der Unterschied zwischen diesen Tags?

Image wird mit Version (1.0.0) und latest getaggt; Version ist fix, latest veränderlich.

b) Welche Informationen finden Sie mit «docker inspect» bei Image und Container, die für spätere Deployments wichtig sind?

docker inspect zeigt wichtige Infos wie Ports, IP, Volumes und Environment-Variablen.

c) Fehlerbild 1: «port is already allocated». Welche Befehle verwenden Sie zur Diagnose und wie beheben Sie das Problem?

netstat -ano | findstr :8080

container stoppen oder anderen port verwenden

d) Fehlerbild 2: «pull access denied» oder «image not found». Welche Ursachen sind wahrscheinlich?

👉 Ursachen:

falscher Image-Name
nicht eingeloggt (docker login)
Image existiert nicht
privates Repository

Fehler „pull access denied“: falscher Name, nicht eingeloggt oder Image existiert nicht.

e) Notieren Sie Ihre persönliche Docker-Debugging-Checkliste mit mindestens sechs Befehlen.

Debugging Befehle:

docker ps, docker logs, docker inspect, docker exec, docker images, docker network ls, docker port.